

Lab 4 Report:

Surpass Human Performance in Fashion MNIST Classification

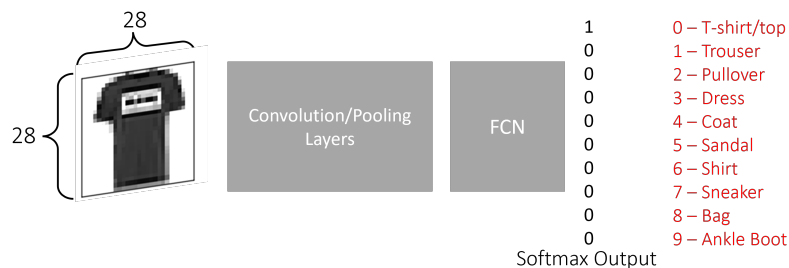
Nate Chaoweeraprasit

```
In [195... %matplotlib inline
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
```

```
In [196... from IPython.display import Image # For displaying images in colab jup
```

```
In [197... Image('lab4_exercise.png', width = 1000)
```

Out[197...  Surpass Human Performance in Fashion MNIST Classification



In this exercise, you will classify fashion item class (28 x 28) using your own **Convolutional Neural Network Architecture**.

Prior to training your neural net, 1) Normalize the dataset using standard scaler and 2) Split the dataset into train/validation/test.

Design your own CNN architecture with your choice of Convolution/Pooling/FCN layers, activation functions, optimization method etc.

Your goal is to **achieve a testing accuracy of >89%**, with no restrictions on epochs (**Human performance: 83.5%**).

Demonstrate the performance of your model via plotting the **training loss**, **validation accuracy** and printing out the **testing accuracy**.

After your model has reached the goal, print the accuracy in each class. What is the class that your model performed the worst?

44

Prepare Data

```
In [198... # Load Fashion-MNIST Dataset in Numpy

device = torch.device("mps" if torch.backends.mps.is_available() else
print(f'\nusing device: {device}\n')

# 10000 training features/targets where each feature is a greyscale im
```

```

train_features = np.load('fashion_mnist_train_features.npy')
train_targets = np.load('fashion_mnist_train_targets.npy')

# 1000 testing features/targets
test_features = np.load('fashion_mnist_test_features.npy')
test_targets = np.load('fashion_mnist_test_targets.npy')

# Let's see the shapes of training/testing datasets
print("Training Features Shape: ", train_features.shape)
print("Training Targets Shape: ", train_targets.shape)
print("Testing Features Shape: ", test_features.shape)
print("Testing Targets Shape: ", test_targets.shape)

```

using device: mps

```

Training Features Shape: (10000, 28, 28)
Training Targets Shape: (10000,)
Testing Features Shape: (1000, 28, 28)
Testing Targets Shape: (1000,)

```

In [199... *# Visualizing the first three training features (samples)*

```

plt.figure(figsize = (10, 10))

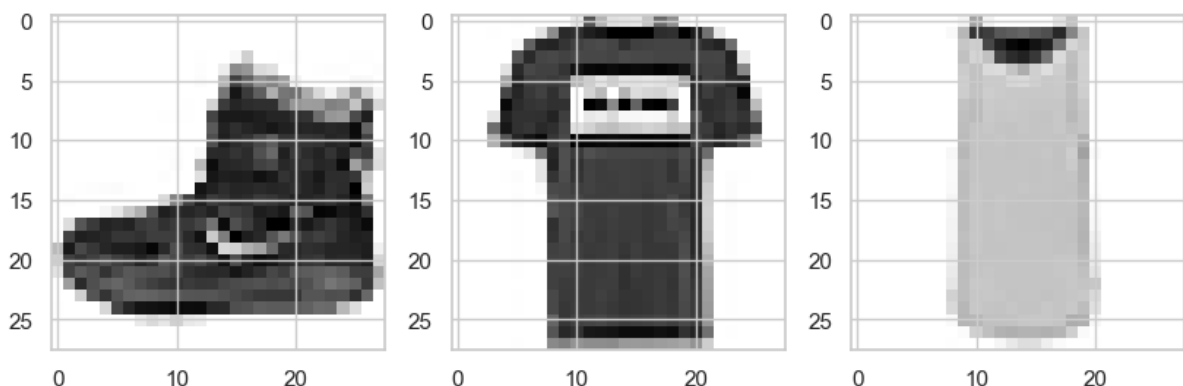
plt.subplot(1,3,1)
plt.imshow(train_features[0], cmap = 'Greys')

plt.subplot(1,3,2)
plt.imshow(train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(train_features[2], cmap = 'Greys')

```

Out[199... <matplotlib.image.AxesImage at 0x140454910>



In [200... *# Reshape features via flattening the images*
This refers to reshape each sample from a 2d array to a 1d array.
hint: np.reshape() function could be useful here

```

train_features = train_features.reshape(train_features.shape[0], -1)
test_features = test_features.reshape(test_features.shape[0], -1)

```

In []: *# Define your scaling function*

```
train_mean = np.mean(train_features, axis=0)
train_std = np.std(train_features, axis=0) + 1e-7 # avoid div by 0

# Use the train stats to scale everything
train_features = (train_features - train_mean) / train_std
test_features = (test_features - train_mean) / train_std
```

In [202... *# Take the first 1000 (or randomly select 1000) training features and*

```
validation_features = train_features[:1000]
validation_targets = train_targets[:1000]

# Take the remaining 9000 training features and targets as training set
train_features = train_features[1000:]
train_targets = train_targets[1000:]
```

In [203... *# Reshape train/validation/test sets to conform to PyTorch's (N, Chann*

```
channels = 1
height = 28
width = 28

train_features = train_features.reshape(train_features.shape[0], channels, height, width)
validation_features = validation_features.reshape(validation_features.shape[0], channels, height, width)
test_features = test_features.reshape(test_features.shape[0], channels, height, width)
```

Define Model

In []: *# Define your CNN architecture here*

```
class CNNModel(torch.nn.Module):

    def __init__(self):

        super(CNNModel, self).__init__()

        # 1st convolution [N, 1, 28, 28]
        self.conv1 = nn.Conv2d(in_channels = 1,
                                out_channels = 64,
                                kernel_size = 3,
                                stride = 1,
                                padding = 1)

        # 1st max pool
        self.maxpool1 = nn.MaxPool2d(kernel_size = 2)

        # 2nd convolution
        self.conv2 = nn.Conv2d(in_channels = 64,
```

```

        out_channels = 128,
        kernel_size = 3,
        stride = 1,
        padding = 1)

    # 2nd max pool
    self.maxpool2 = nn.MaxPool2d(kernel_size = 2)

    # fully connected layers
    self.fc1 = nn.Linear(in_features = 128*7*7, out_features = 256)
    self.fc2 = nn.Linear(in_features = 256, out_features = 10)
    self.dropout = nn.Dropout(0.2)
    self.relu = nn.ReLU()

def forward(self, x):

    # YOUR CODE HERE
    # Note: If you are using CrossEntropyLoss() do NOT apply softmax
    # since it's incorporated within the loss function

    # x starts as [N, 1, 28, 28]

    # convolutional layer 1 + max pool
    out = self.conv1(x)
    out = self.relu(out)
    out = self.maxpool1(out)

    # convolutional layer 2 + max pool
    out = self.conv2(out)
    out = self.relu(out)
    out = self.maxpool2(out)

    # flatten
    out = out.view(out.size(0), -1)

    # fully connected layers
    out = self.fc1(out)
    out = self.relu(out)
    out = self.dropout(out)
    out = self.fc2(out)

    return out

```

Select Hyperparameters

```

In [205... # Fix the random seed so that model performance is reproducible
torch.manual_seed(55)

# Initialize your CNN model

```

```

model = CNNModel().to(device)

# Define learning rate, epoch and batchsize for mini-batch gradient

learning_rate = 0.0008
epochs = 35
batchsize = 64

# Define loss function and optimizer

loss_func = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

model

```

```

Out[205... CNNModel(
  (conv1): Conv2d(1, 64, kernel_size=(3, 3), stride=(1, 1), padding=(
1, 1))
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=
1, ceil_mode=False)
  (conv2): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding
=(1, 1))
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=
1, ceil_mode=False)
  (fc1): Linear(in_features=6272, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=10, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
  (relu): ReLU()
)

```

Identify Tracked Values

```

In [206... # Placeholders for training loss and validation accuracy during traini
# Training loss should be tracked for each iteration (1 iteration -> s
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> fu
# If using batch gradient, 1 iteration = 1 epoch

train_loss_list = []
validation_accuracy_list = np.zeros((epochs,))

```

Train Model

```

In [207... import tqdm # Use "for epoch in tqdm.trange(epochs):" to see the progr

# Convert the training, validation, testing dataset (NumPy arrays) int
# Split your training features/targets into mini-batches if using mini

train_inputs = torch.from_numpy(train_features).float().to(device)
train_targets = torch.from_numpy(train_targets).long().to(device)

```

```

validation_inputs = torch.from_numpy(validation_features).float().to(device)
validation_targets = torch.from_numpy(validation_targets).long().to(device)

testing_inputs = torch.from_numpy(test_features).float().to(device)
testing_targets = torch.from_numpy(test_targets).long().to(device)

train_batches_features = torch.split(train_inputs, batchsize)
train_batches_targets = torch.split(train_targets, batchsize)

# length of train_batches_features = total number of mini-batches in training set
batch_split_num = len(train_batches_features)

# Training Loop -----

for epoch in range(epochs):

    # For each mini-batch number i, grab i-th training feature mini-batch
    # and perform fwd/bwd pass on the network

    for batch_num in range(batch_split_num):
        optimizer.zero_grad()
        train_batch_outputs = model(train_batches_features[batch_num])
        loss = loss_func(train_batch_outputs, train_batches_targets[batch_num])
        train_loss_list.append(loss.item())
        loss.backward()
        optimizer.step()

    # Compute Validation Accuracy -----
    with torch.no_grad():
        validation_outputs = model(validation_inputs)
        correct = (torch.argmax(validation_outputs, dim=1) ==
                    validation_targets).type(torch.FloatTensor)

        print(f"epoch {epoch+1:02d} | val accuracy: {correct.mean():.3f}")

        validation_accuracy_list[epoch] = correct.mean()

```

```
epoch 01 | val accuracy: 0.847
epoch 02 | val accuracy: 0.863
epoch 03 | val accuracy: 0.883
epoch 04 | val accuracy: 0.878
epoch 05 | val accuracy: 0.880
epoch 06 | val accuracy: 0.888
epoch 07 | val accuracy: 0.872
epoch 08 | val accuracy: 0.877
epoch 09 | val accuracy: 0.861
epoch 10 | val accuracy: 0.881
epoch 11 | val accuracy: 0.890
epoch 12 | val accuracy: 0.899
epoch 13 | val accuracy: 0.901
epoch 14 | val accuracy: 0.888
epoch 15 | val accuracy: 0.898
epoch 16 | val accuracy: 0.904
epoch 17 | val accuracy: 0.899
epoch 18 | val accuracy: 0.893
epoch 19 | val accuracy: 0.904
epoch 20 | val accuracy: 0.887
epoch 21 | val accuracy: 0.901
epoch 22 | val accuracy: 0.907
epoch 23 | val accuracy: 0.899
epoch 24 | val accuracy: 0.904
epoch 25 | val accuracy: 0.898
epoch 26 | val accuracy: 0.901
epoch 27 | val accuracy: 0.878
epoch 28 | val accuracy: 0.907
epoch 29 | val accuracy: 0.910
epoch 30 | val accuracy: 0.889
epoch 31 | val accuracy: 0.888
epoch 32 | val accuracy: 0.905
epoch 33 | val accuracy: 0.885
epoch 34 | val accuracy: 0.896
epoch 35 | val accuracy: 0.891
```

Visualize & Evaluate Model

```
In [208... # Seaborn for prettier plot

import seaborn as sns

sns.set(style = 'whitegrid', font_scale = 1)
```

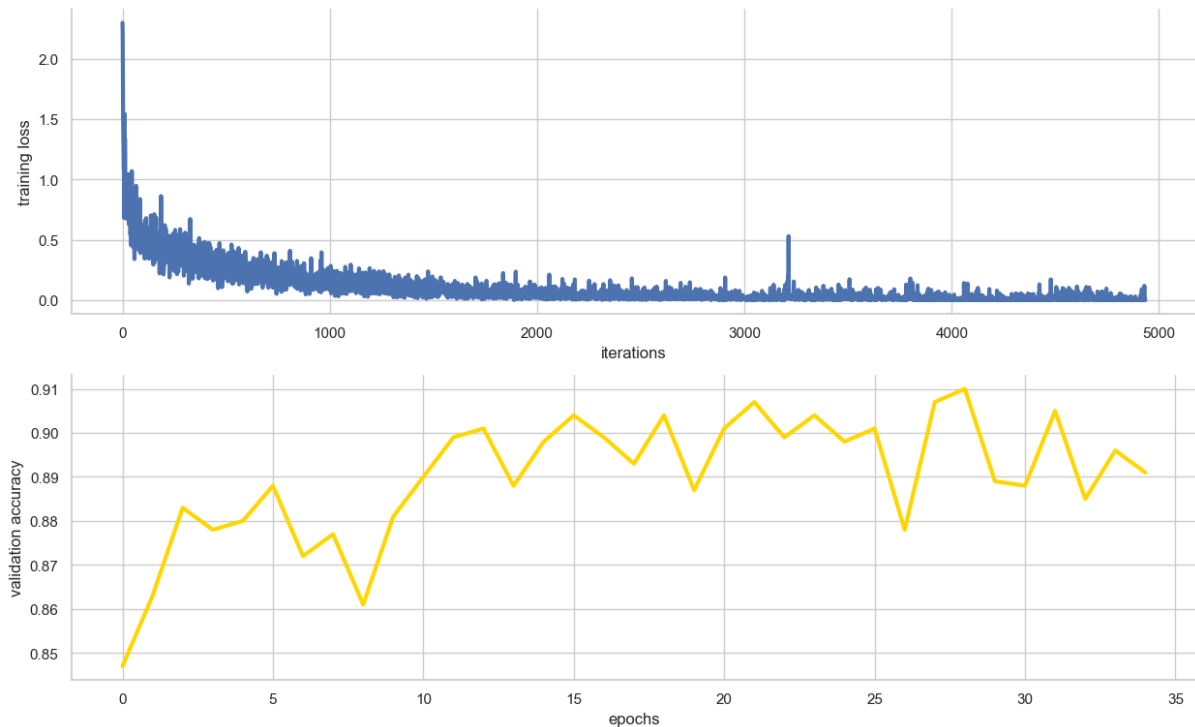
```
In [209... # Visualize training loss

plt.figure(figsize = (15, 9))

plt.subplot(2, 1, 1)
plt.plot(train_loss_list, linewidth = 3)
plt.xlabel("iterations")
```

```
plt.ylabel("training loss")
sns.despine()

plt.subplot(2, 1, 2)
plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
plt.xlabel("epochs")
plt.ylabel("validation accuracy")
sns.despine()
```



In [210... *# Compute the testing accuracy*

```
with torch.no_grad():

    testing_outputs = model(testing_inputs)
    prediction = torch.argmax(testing_outputs, dim=1)
    correct = (prediction == testing_targets).type(torch.FloatTensor)

    print(f"Testing Accuracy: {correct.mean():.3f}")
```

Testing Accuracy: 0.892